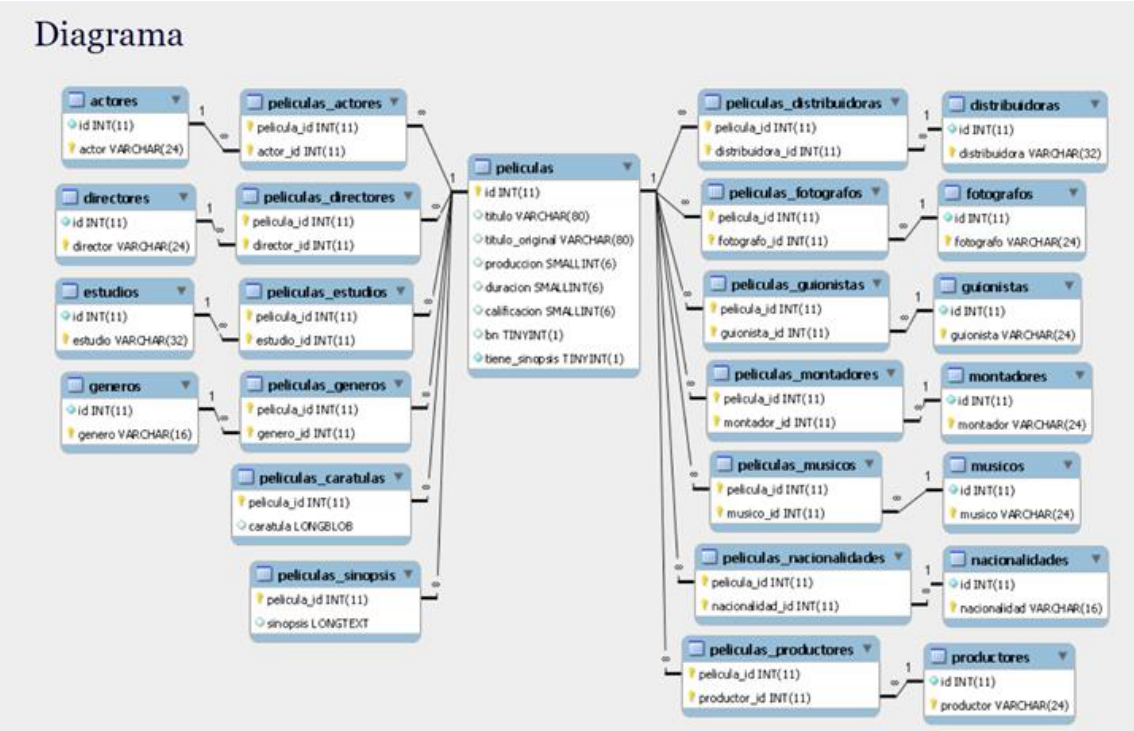
BOLETÍN DE EJERCICIOS TEMA 3

PROFESOR: MARTÍN GARCÍA FIGUEIRA
martingarcia@fpfomento.com

NOTA: La resolución de cada uno de los ejercicios debe ser una captura de pantalla en la que se refleje la consulta y los resultados obtenidos de ella. Debéis usar la base de datos filmoteca, que os he subido junto al boletín. El diagrama de dicha base es:



Ejercicio 1

Actualiza la tabla **nacionalidades** de modo que España aparezca en mayúsculas. Muestra la sentencia SQL y el resultado.

Solución

```
2 SET nacionalidad=UPPER(nacionalidad)
3 WHERE nacionalidad='España';
4
5 SELECT * FROM nacionalidades
6
```

nacionalidades (2×32)

id	nacionalidad
1	Alemania
2	Estados Unidos
3	Francia
4	Gran Bretaña
5	ESPAÑA
6	Checoslovaquia
7	Italia
8	Japón

Ejercicio 2

Obtener un listado de los directores y actores ordenándolos por orden descendente del número de películas en las que han participado. En el listado debe figurar en una columna el nombre del actor o director, en otra el número de películas, en otra si es director o autor y en la última el número de letras que contienen sus nombres (sin apellidos).

Solución

```sql
SELECT director, COUNT(*) cuenta, 'DIRECTOR' tipo,
CASE
WHEN  POSITION(' ' IN director) =0 THEN LENGTH(director)
ELSE POSITION(' ' IN director)-1
END AS 'Longitud'
FROM directores_peliculas
GROUP BY 1,3
UNION
SELECT actor, COUNT(*), 'ACTOR' tipo,
CASE
WHEN POSITION(' ' IN actor) =0 THEN LENGTH(actor)
ELSE POSITION(' ' IN actor)-1
END AS 'Longitud'
FROM actores_peliculas
GROUP BY 1,3
ORDER BY 2 DESC, 1
```

Resultado #1 (4×5.969)

director	cuenta	tipo	Longitud
Zhang Yi	1	ACTOR	5
Zhao Xiouri	1	ACTOR	4
Zhu Qanqing	1	ACTOR	3
Zita Johann	1	ACTOR	4
Zoe Nathanson	1	ACTOR	3
Zofia Mrozowska	1	ACTOR	5
Zouzou	1	ACTOR	6
Zsa Zsa Gabor	1	ACTOR	3
Zygmunt Malanowicz	1	ACTOR	7

Ejercicio 3

Teniendo en cuenta que para dividir se utiliza "/", para multiplicar "*" y recordando funciones de redondeo que hemos visto, averiguar, según los datos que tenemos en nuestra base de datos de cine, cuántas películas se hicieron en cada década mostrando número de películas y década. Si hay películas sin fecha en su lugar debe poner 'Desconocida'.

Solución:

Host: 127.0.0.1 | Base de datos: filmoteca | Consulta | Consulta #2*

```sql
SELECT COALESCE(FLOOR(produccion/10)*10,'DESCONOCIDA') AS DECADA, COUNT(*)
FROM peliculas
GROUP BY 1
ORDER BY 1
```

Resultado #1 (2×10)

DECADA	COUNT(*)
1910	29
1920	63
1930	238
1940	424
1950	546
1960	467
1970	459
1980	642
1990	467
DESCONOCIDA	3

Ejercicio 4

Mostrar en dos columnas, **Letra** y **Nº de películas**, el número de películas que comienzan por cada letra del abecedario. Las películas que no empiecen por una letra se deben agrupar en un grupo "**No letra**".

Solución:

```sql
CREATE VIEW titulo_letraabecedario AS
   SELECT
      CASE WHEN SUBSTRING(titulo,1,1) BETWEEN 'A' AND 'Z'
         THEN SUBSTRING(titulo,1,1)
         ELSE 'No letra'
      END inicial
   FROM peliculas;

SELECT inicial, COUNT(*) cuenta
FROM titulo_letraabecedario
GROUP BY 1
ORDER BY 1
```

Resultado #1 (2×26)

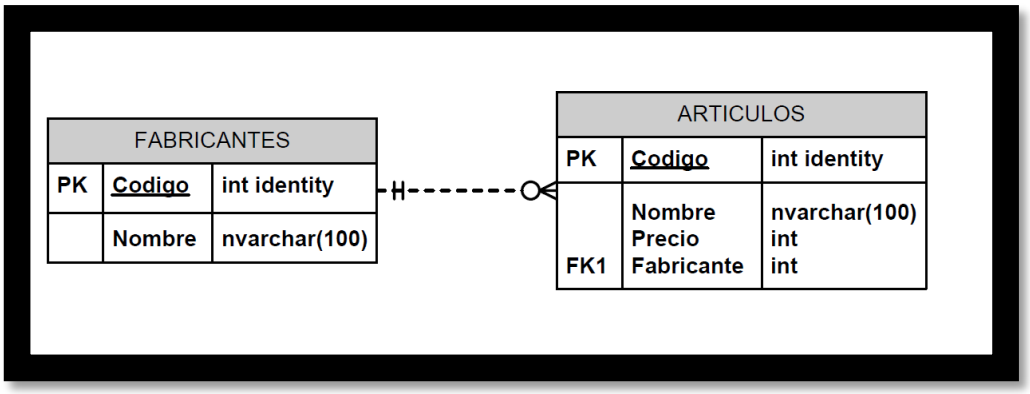
inicial	cuenta
A	186
B	95
C	246
D	128
E	702
F	57
G	55
H	67
I	28

EJERCICIOS BASE DE DATOS BDMONTECASTELO

NOTA: Para hacer estos ejercicios debemos importar el script **BDMontecastelo.sql**

1. **<u>La Tienda de Informática</u>**

   Haz las siguientes consultas:



1.17. Añadir un nuevo artículo: Altavoces de 70 € (del fabricante 2)

```
INSERT INTO ARTICULOS( Nombre , Precio , Fabricante)
    VALUES ( 'Altavoces' , 70 , 2 )
```

1.18. Cambiar el nombre del artículo 8 a 'Impresora Laser'

```
UPDATE ARTICULOS
        SET Nombre = 'Impresora Laser'
    WHERE Codigo = 8
```

1.19. Aplicar un descuento del 10% (multiplicar el precio por 0'9) a todos los productos.

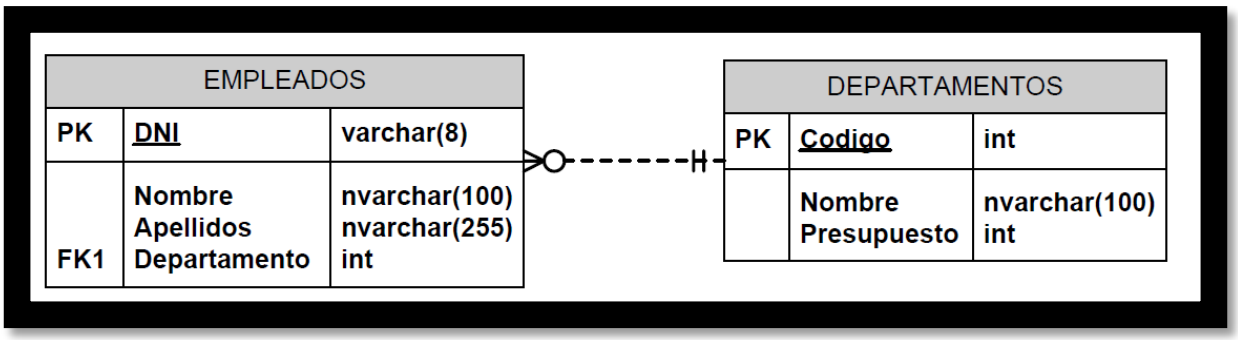
```
UPDATE ARTICULOS
SET Precio = Precio * 0.9
```

1.20. Aplicar un descuento de 10 € a todos los productos cuyo precio sea mayor o igual a 120 €

```
UPDATE ARTICULOS
        SET Precio = Precio - 10
    WHERE Precio >= 120
```

2. Empleados

Haz las siguientes consultas:



2.15. Añadir un nuevo departamento: 'Calidad', con presupuesto de 40.000 € y código 11. Añadir un empleado vinculado al departamento recién creado: Esther Vázquez, DNI: 89267109

```
INSERT INTO DEPARTAMENTOS
    VALUES ( 11 , 'Calidad' , 40000)

INSERT INTO EMPLEADOS
    VALUES ( '89267109' , 'Esther' , 'Vázquez' , 11)
```

2.16. Aplicar un recorte presupuestario del 10% a todos los departamentos.

```
UPDATE DEPARTAMENTOS SET Presupuesto = Presupuesto * 0.9
```

2.17. Reasignar a los empleados del departamento de investigación (código 77) al departamento de informática (código 14).

```
UPDATE EMPLEADOS SET Departamento = 14 WHERE Departamento = 77
```

2.18. Despedir a todos los empleados que trabajan para el departamento de informática (código 14).

```
DELETE FROM EMPLEADOS
WHERE Departamento = 14
```

2.19. Despedir a todos los empleados que trabajen para departamentos cuyo presupuesto sea superior a los 60.000 ¤.

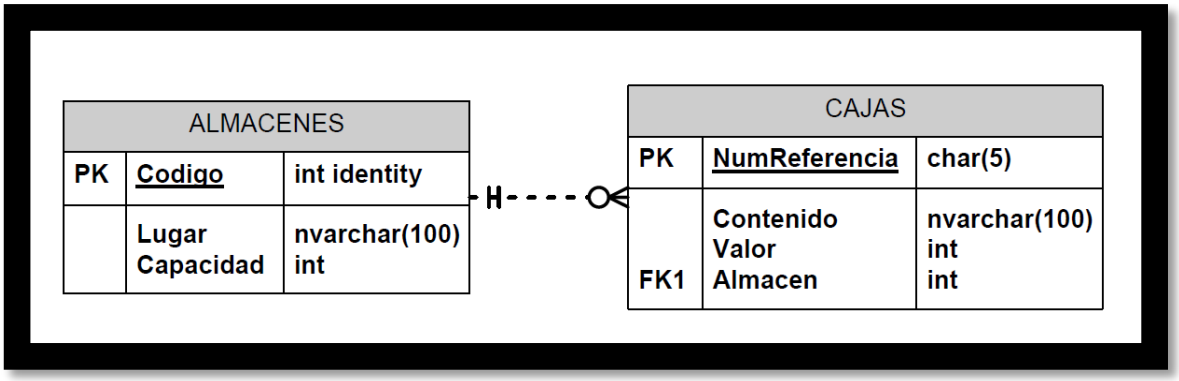
```
DELETE FROM EMPLEADOS
   WHERE Departamento IN
   (
      SELECT Codigo FROM DEPARTAMENTO
         WHERE Presupuesto >= 60000
   )
```

2.20. Despedir a todos los empleados.

```
DELETE FROM EMPLEADOS
```

3. Almacenes

Haz las siguientes consultas:



3.1 Insertar un nuevo almacén en Barcelona con capacidad para 3 cajas.

```
INSERT INTO ALMACENES(Lugar,Capacidad) VALUES('Barcelona',3)
```

3.6. Insertar una nueva caja, con número de referencia 'H5RT', con contenido 'Papel', valor 200, y situada en el almacén 2.

```
INSERT INTO CAJAS
VALUES('H5RT','Papel',200,2)
```

3.7. Rebajar el valor de todas las cajas un 15%.

```
UPDATE CAJAS SET Valor = Valor * 0.85
```

3.8. Rebajar un 20% el valor de todas las cajas cuyo valor sea superior al valor medio de todas las cajas.

```
UPDATE CAJAS SET Valor = Valor * 0.80
   WHERE Valor > (SELECT AVG(Valor) FROM CAJAS)
```

3.9. Eliminar todas las cajas cuyo valor sea inferior a 100 ¤.

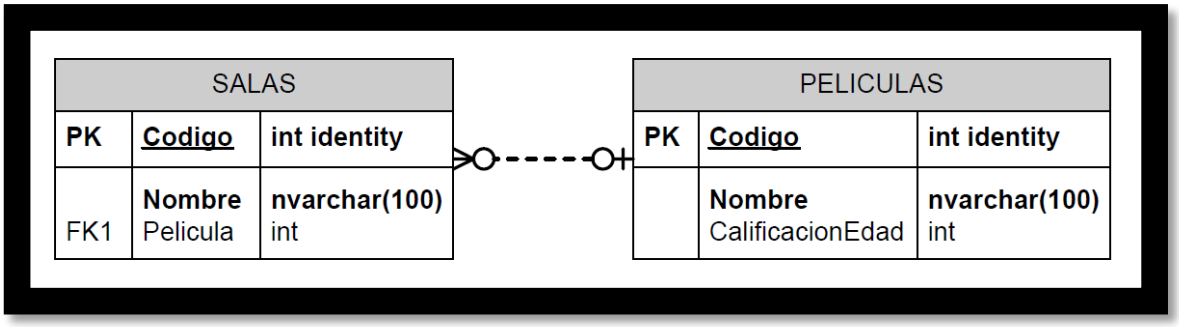
```
DELETE FROM CAJAS WHERE Valor < 100
```

3.10. Vaciar el contenido de los almacenes que están saturados.

```
DELETE FROM CAJAS WHERE Almacen IN
  (
    SELECT Codigo
      FROM ALMACENES
      WHERE Capacidad < (
        SELECT COUNT(*)
          FROM CAJAS
          WHERE Almacen = Codigo
      )
  )
```

4. Películas y Salas

Haz las siguientes consultas:



4.6. Añadir una nueva película 'Uno, Dos, Tres', para mayores de 7 años.

```
INSERT INTO PELICULAS(Nombre,CalificacionEdad) VALUES('Uno, Dos, Tres',7)
```

4.7. Hacer constar que todas las películas no calificadas han sido calificadas 'no recomendables para menores de 13 años'.

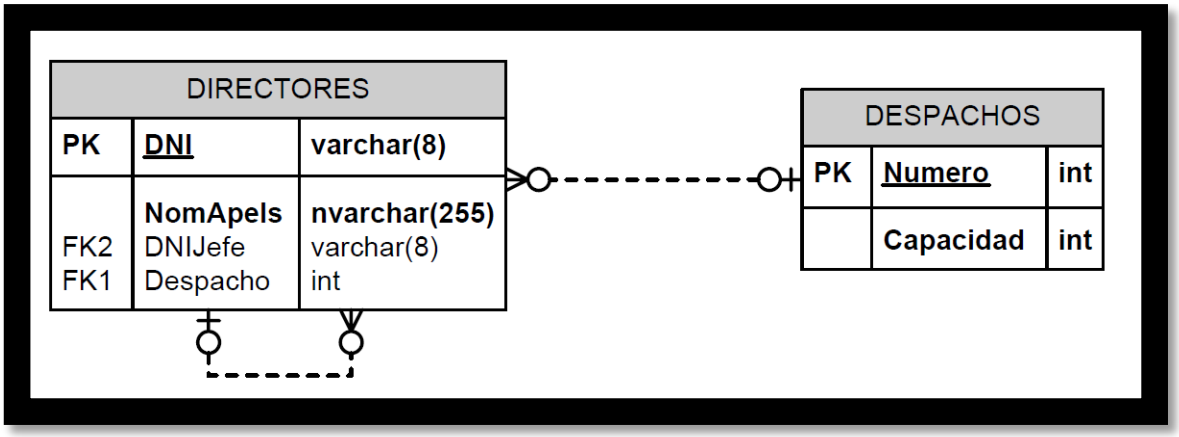
```
UPDATE PELICULAS SET CalificacionEdad=13 WHERE CalificacionEdad IS NULL
```

4.8. Eliminar todas las salas que proyectan películas recomendadas para todos los públicos.

```
DELETE FROM SALAS WHERE Pelicula IN
    (SELECT Codigo FROM PELICULAS WHERE CalificacionEdad = 0)
```

5. Directores

Haz las siguientes consultas:



5.5. Añadir un nuevo director llamado Paco Pérez, DNI 28301700, sin jefe, y situado en el despacho 124.

```
INSERT INTO DIRECTORES VALUES('28301700', 'Paco P´erez', NULL, 124)
```

5.6. Asignar a todos los empleados apellidados Pérez un nuevo jefe con DNI 74568521.

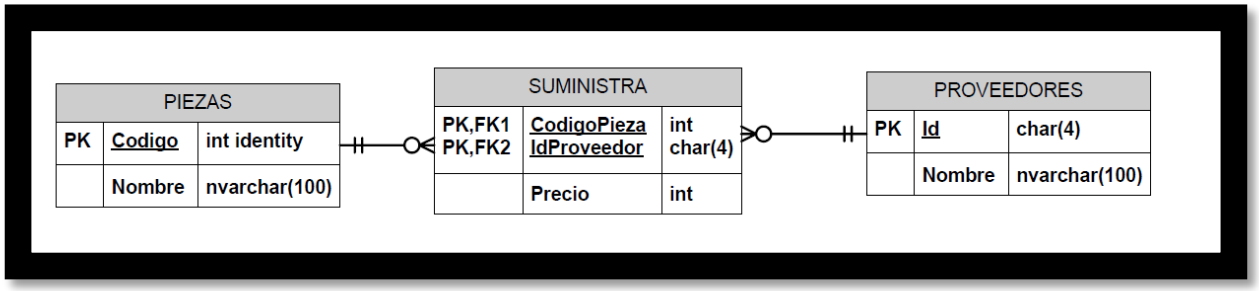
```
UPDATE DIRECTORES SET DNIJefe = '74568521' WHERE NomApels LIKE '%P´erez%'
```

5.7. Despedir a todos los directores, excepto a los que no tienen jefe.

```
DELETE FROM DIRECTORES WHERE DNIJefe IS NOT NULL
```

6. Piezas y Proveedores

Haz las siguientes consultas:



6.7. Hacer constar en la base de datos que la empresa ”Skellington Supplies”(codigo TNBC) va a empezar a suministrarnos tuercas (código 1) a 7 pesetas cada tuerca.

```
INSERT INTO SUMINISTRA VALUES ( 'TNBC'
  , 1 , 7 )
```

6.8. Aumentar los precios en una unidad.

```
UPDATE SUMINISTRA SET Precio = Precio + 1
```

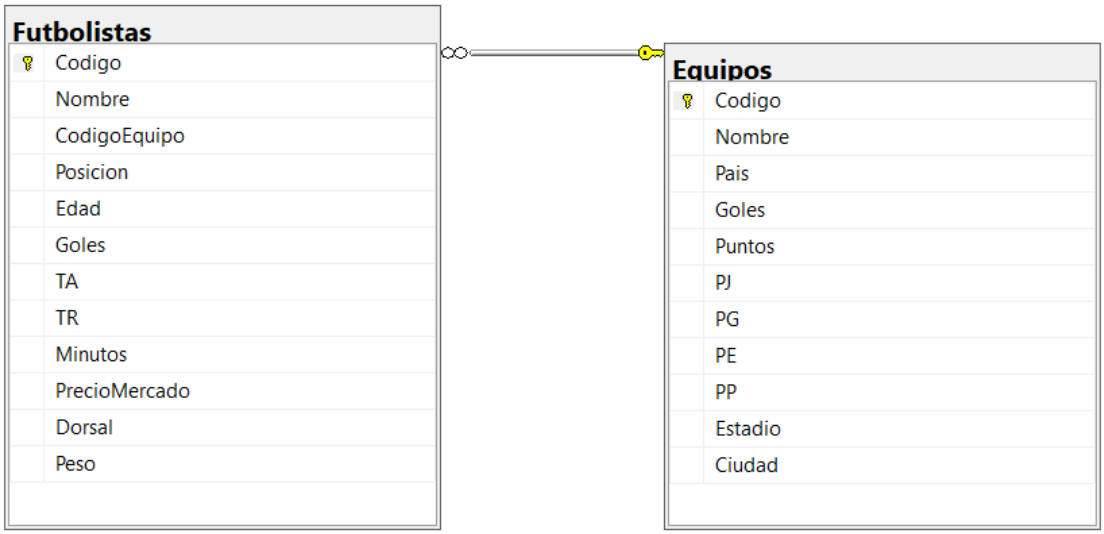
6.9. Hacer constar en la base de datos que la empresa ”Susan Calvin Corp.”(RBT) no va a suministrarnos ninguna pieza (aunque la empresa en s´ı va a seguir constando en nuestra base de datos).

```
DELETE FROM SUMINISTRA
WHERE IdProveedor = 'RBT'
```

6.10. Hacer constar en la base de datos que la empresa ”Susan Calvin Corp.”(RBT) ya no va a suministrarnos clavos (código 4)

```
DELETE FROM SUMINISTRA
WHERE IdProveedor = 'RBT'
AND CodigoPieza = 4
```

EJERCICIOS BASE DE DATOS FUTBOL



Bloque 1: Funciones y DML (Insert, Update, Delete)

1. Inserta un nuevo jugador en la tabla Futbolistas especificando solo el nombre, apellido y equipo. Deja que el resto de columnas tomen sus valores por defecto.

```
INSERT INTO Futbolistas (Codigo, Nombre, CodigoEquipo)
VALUES ('NUEVO01', 'Juan Pérez', '#FCBLNA');
```

2. Actualiza la tabla Equipos eliminando los espacios en blanco sobrantes al principio y al final de todos los nombres de los estadios usando TRIM.

```
UPDATE Equipos SET Estadio = TRIM(Estadio);
```

3. Realiza una consulta que muestre el nombre del jugador y su apellido. Si el apellido es NULL, debe mostrar "Sin apellido" usando ISNULL o COALESCE.

```
--3
SELECT
    Nombre AS Nombre_Original,
    -- Si hay un espacio, pillamos lo de la derecha. Si no, devolvemos NULL.
    CASE
        WHEN CHARINDEX(' ', Nombre) > 0 THEN SUBSTRING(Nombre, CHARINDEX(' ', Nombre) + 1, LEN(Nombre))
        ELSE NULL
    END AS Apellido_Extraido
INTO #TempApellidos -- Lo metemos en una tabla temporal para la prueba
FROM Futbolistas;

-- Ahora aplicamos el ISNULL sobre la extracción
SELECT
    Nombre_Original,
    COALESCE(Apellido_Extraido, 'Sin apellido') AS Resultado
FROM #TempApellidos;
```

4. Elimina a todos los jugadores que pertenezcan a un equipo que ya no existe en la tabla Equipos (Usa una subconsulta).

```
DELETE FROM Futbolistas
WHERE CodigoEquipo NOT IN (SELECT Codigo FROM Equipos);
```

5. Incrementa los goles de todos los equipos en un 15% si la cantidad que han marcado es inferior a la media.

```
UPDATE Equipos
SET Goles = Goles * 1.15
WHERE Goles < (SELECT AVG(Goles) FROM Equipos);
```

6. Muestra el nombre de los equipos en minúsculas y su código de ciudad en mayúsculas.

```
SELECT LOWER(Nombre) AS EquipoMin, UPPER(Ciudad) AS CiudadMay
FROM Equipos;
```

7. Crea una tabla llamada Historico_Goles e inserta en ella todos los jugadores que hayan marcado más de 50 goles

```
SELECT * INTO Historico_Goles
FROM Futbolistas WHERE Goles > 50;
```

8. Muestra el nombre de los jugadores y la raíz cuadrada de sus goles totales, redondeando el resultado hacia arriba (CEILING).

```
SELECT Nombre, CEILING(SQRT(Goles)) AS RaizGoles
FROM Futbolistas;
```

9. Crea una consulta que inserte en una tabla de auditoría el nombre del usuario actual del sistema (SYSTEM_USER) y la hora de la operación cada vez que se consulte la tabla de salarios.

```
INSERT INTO Auditoria (Usuario, FechaOperacion, Accion)
VALUES (SYSTEM_USER, CURRENT_TIMESTAMP, 'Consulta Salarios');
```

Bloque 2: Sentencia MERGE

1. Tienes una tabla #NuevosFichajes. Usa MERGE para insertarlos en Futbolistas si no existen y actualizar sus goles si ya están.

```
MERGE Futbolistas AS Destino
USING #NuevosFichajes AS Origen
ON Destino.Codigo = Origen.Codigo
WHEN MATCHED THEN
    UPDATE SET Destino.Goles = Origen.Goles
WHEN NOT MATCHED THEN
    INSERT (Codigo, Nombre, CodigoEquipo, Goles)
    VALUES (Origen.Codigo, Origen.Nombre, Origen.CodigoEquipo, Origen.Goles);
```

2. Si un jugador está en la tabla Futbolistas pero NO aparece en la tabla de Inscritos_Temporada, elimínalo usando WHEN NOT MATCHED BY SOURCE.

```
MERGE Futbolistas AS Destino
USING Inscritos_Temporada AS Origen
ON Destino.Codigo = Origen.CodigoJugador
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;
```

3. Realiza un MERGE donde solo actualices el precio de mercado del jugador si el nuevo precio de mercado que viene de la tabla origen es distinto al actual.

```
MERGE Futbolistas AS Destino
USING #NuevosFichajes AS Origen
ON Destino.Codigo = Origen.Codigo
WHEN MATCHED AND Origen.PrecioMercado <> Destino.PrecioMercado THEN
    UPDATE SET Destino.PrecioMercado = Origen.PrecioMercado;
```

4. Implementa un MERGE que inserte un registro si no coincide por ID, pero que solo lo haga si el jugador tiene más de 18 años.

```
MERGE Futbolistas AS Destino
USING #NuevosFichajes AS Origen
ON Destino.Codigo = Origen.Codigo
WHEN NOT MATCHED AND Origen.Edad > 18 THEN
    INSERT (Codigo, Nombre, Edad, CodigoEquipo)
    VALUES (Origen.Codigo, Origen.Nombre, Origen.Edad, Origen.CodigoEquipo);
```

5. Sincroniza la tabla Estadisticas basándote en un MERGE que compare tanto el ID_Jugador como el ID_Temporada.

```
MERGE Estadisticas AS Destino
USING #EstadisticasNuevas AS Origen
ON Destino.ID_Jugador = Origen.ID_Jugador AND Destino.ID_Temporada = Origen.ID_Temporada
WHEN MATCHED THEN
    UPDATE SET Destino.Goles = Origen.Goles
WHEN NOT MATCHED THEN
    INSERT (ID_Jugador, ID_Temporada, Goles)
    VALUES (Origen.ID_Jugador, Origen.ID_Temporada, Origen.Goles);
```

6. Si un equipo ya existe en la tabla destino, suma sus puntos actuales con el nuevo que llega de la tabla origen. Si no existe, insértalo.

```
MERGE Equipos AS Destino
USING #NuevosPuntosEquipos AS Origen
ON Destino.Codigo = Origen.Codigo
WHEN MATCHED THEN
    UPDATE SET Destino.Puntos = Destino.Puntos + Origen.Puntos
WHEN NOT MATCHED THEN
    INSERT (Codigo, Nombre, Puntos)
    VALUES (Origen.Codigo, Origen.Nombre, Origen.Puntos);
```

7. Si un jugador coincide en el MERGE pero su estado en la tabla origen es 'Retirado', bórralo de la tabla destino en lugar de actualizarlo.

```sql
MERGE Futbolistas AS Destino
USING #CargaJugadores AS Origen
ON Destino.Codigo = Origen.Codigo
WHEN MATCHED AND Origen.Estado = 'Retirado' THEN
    DELETE
WHEN MATCHED THEN
    UPDATE SET Destino.Goles = Origen.Goles;
```

8. Crea un MERGE que actualice la tabla Equipos basándose en una vista que une datos de varias ciudades.

```sql
MERGE Equipos AS Destino
USING VistaCiudadesEquipos AS Origen
ON Destino.Codigo = Origen.Codigo
WHEN MATCHED THEN
    UPDATE SET Destino.Pais = Origen.Pais;
```

9. Usa MERGE para actualizar la posición de los equipos en la tabla Liga basándote en una tabla temporal de resultados recién calculados.

```sql
MERGE Liga AS Destino
USING #ResultadosCalculados AS Origen
ON Destino.CodigoEquipo = Origen.CodigoEquipo
WHEN MATCHED THEN
    UPDATE SET Destino.Posicion = Origen.Posicion,
               Destino.Puntos = Origen.Puntos
WHEN NOT MATCHED THEN
    INSERT (CodigoEquipo, Posicion, Puntos)
    VALUES (Origen.CodigoEquipo, Origen.Posicion, Origen.Puntos);
```

Bloque 3: Transacciones (BEGIN, COMMIT, ROLLBACK)

1. Inicia una transacción para cambiar a un jugador de equipo. Debes hacer un UPDATE en la tabla Futbolistas y otro UPDATE en la tabla Equipos para aumentar el número de goles del equipo con los goles del futbolista. Si uno falla, deshazlo.

```
--1
BEGIN TRANSACTION;

-- Declaramos variables para el ejemplo
DECLARE @CodigoJugador VARCHAR(50) = 'VL0021'; -- Ejemplo: Gameiro
DECLARE @NuevoEquipo VARCHAR(50) = '#RMDRID';
DECLARE @GolesJugador INT;

-- 1. Obtenemos los goles que tiene el jugador actualmente
SELECT @GolesJugador = Goles FROM Futbolistas WHERE Codigo = @CodigoJugador;

-- 2. Cambiamos al jugador de equipo
UPDATE Futbolistas
SET CodigoEquipo = @NuevoEquipo
WHERE Codigo = @CodigoJugador;

-- 3. Aumentamos los goles del equipo destino con los del futbolista
UPDATE Equipos
SET Goles = Goles + @GolesJugador
WHERE Codigo = @NuevoEquipo;

-- 4. Verificamos si hubo algún error en los pasos anteriores
IF @@ERROR <> 0
    BEGIN
        PRINT 'Error detectado. Deshaciendo cambios...';
        ROLLBACK; -- Cancela toda la operación
    END
ELSE
    BEGIN
        PRINT 'Traspaso y actualización de goles completada con éxito.';
        COMMIT; -- Confirma los cambios permanentemente
    END
```

2. Inicia una transacción, borra todos los datos de la tabla Goles y luego ejecuta un ROLLBACK. Verifica que los datos siguen ahí.

```
BEGIN TRANSACTION;
    DELETE FROM Futbolistas; -- Borramos datos

    -- Comprobamos que está vacía
    SELECT COUNT(*) AS Filas_Ahora FROM Futbolistas;

ROLLBACK; -- Deshacemos todo

-- Comprobamos que los datos han vuelto
SELECT COUNT(*) AS Filas_Restauradas FROM Futbolistas;
```

3. Inserta 3 registros en la tabla Ciudades. Asegúrate de que si la tercera inserción falla, las dos primeras tampoco se guarden.

```
BEGIN TRANSACTION;
    INSERT INTO Ciudades (NombreCiudad, Pais) VALUES ('Sevilla', 'España');
    INSERT INTO Ciudades (NombreCiudad, Pais) VALUES ('Bilbao', 'España');
    -- La tercera fallará si el nombre es demasiado largo o el tipo de dato no coincide
    INSERT INTO Ciudades (NombreCiudad, Pais) VALUES ('CiudadConNombreExtremadamenteLargoParaQueFalle', 'España');

IF @@ERROR <> 0
    ROLLBACK; -- Si falla la última, no se guardan las dos primeras
ELSE
    COMMIT;
```

4. Crea una transacción que aumente los goles de los delanteros, pero que haga ROLLBACK si el total de goles tras la subida supera una cantidad variable que hayas establecido.

```
-- 1. Definimos la cantidad variable (umbral máximo de goles permitidos para delanteros)
DECLARE @MaxGolesPermitidos INT = 500;
DECLARE @TotalGolesDespues INT;

BEGIN TRANSACTION;
    -- 2. Aumentamos los goles de los delanteros (por ejemplo, sumamos 2 a cada uno)
    UPDATE Futbolistas
    SET Goles = Goles + 2
    WHERE Posicion = 'Delantero';

    -- 3. Calculamos el nuevo total de goles de todos los delanteros
    SELECT @TotalGolesDespues = SUM(Goles)
    FROM Futbolistas
    WHERE Posicion = 'Delantero';

    -- 4. Verificamos si superamos el límite establecido
    IF @TotalGolesDespues > @MaxGolesPermitidos
        BEGIN
            PRINT 'Límite excedido (' + CAST(@TotalGolesDespues AS VARCHAR) + ' goles). Deshaciendo cambios...';
            ROLLBACK; -- Cancela la subida de goles
        END
    ELSE
        BEGIN
            PRINT 'Subida confirmada. Nuevo total: ' + CAST(@TotalGolesDespues AS VARCHAR);
            COMMIT; -- Guarda los cambios permanentemente
        END
```

5. Simula un proceso de fin de temporada: mueve los datos a la tabla Historico, vacía la tabla Actual y confirma los cambios solo si el conteo de filas coincide.

```sql
DECLARE @FilasOriginales INT;
DECLARE @FilasInsertadas INT;

SELECT @FilasOriginales = COUNT(*) FROM Futbolistas;

BEGIN TRANSACTION;
    -- 1. Copiamos a la tabla histórico
    INSERT INTO Historico_Goles SELECT * FROM Futbolistas;
    SET @FilasInsertadas = @@ROWCOUNT;

    -- 2. Vaciamos la actual
    DELETE FROM Futbolistas;

IF @FilasOriginales = @FilasInsertadas
    COMMIT;
ELSE
    ROLLBACK;
```

6. Inicia una transacción para borrar el VALENCIA. Primero debes borrar todos sus jugadores y luego el equipo. Si el borrado de jugadores falla, el equipo no debe borrarse.

```sql
BEGIN TRANSACTION;
    -- Primero los hijos (jugadores)
    DELETE FROM Futbolistas WHERE CodigoEquipo = '#VLNCIA';

    -- Luego el padre (equipo)
    DELETE FROM Equipos WHERE Codigo = '#VLNCIA';

IF @@ERROR <> 0
    ROLLBACK;
ELSE
    COMMIT;
```

7. Crea una consulta con un BEGIN TRANSACTION que inserte datos, pero "olvida" poner el COMMIT. Abre otra ventana de consulta e intenta leer la tabla. ¿Qué sucede?

```sql
-- Ejecuta esto y NO ejecutes nada más
BEGIN TRANSACTION;
    INSERT INTO Futbolistas (Codigo, Nombre) VALUES ('TEMP01', 'Jugador Fantasma');

-- Abre otra pestaña y haz "SELECT * FROM Futbolistas".
```

8. Realiza una transacción que actualice los puntos de los equipos tras una jornada. Si la suma total de puntos otorgados no es igual a (Partidos * 3), haz ROLLBACK.

```
DECLARE @PartidosJugados INT = 10;
DECLARE @PuntosEsperados INT = @PartidosJugados * 3;
DECLARE @PuntosRepartidos INT;

BEGIN TRANSACTION;
    UPDATE Equipos SET Puntos = Puntos + 3 WHERE Codigo IN ('#RMDRID', '#FCBLNA'); -- Ejemplo

    SELECT @PuntosRepartidos = SUM(Puntos) FROM Equipos; -- Simplificación para el ejercicio

IF @PuntosRepartidos <> @PuntosEsperados -- Lógica de control
    ROLLBACK;
ELSE
    COMMIT;
```

Bloque 4: TRY-CATCH

1. Intenta insertar un registro con una Clave Primaria que ya existe. Usa TRY-CATCH para que en lugar de un error rojo de SQL, devuelva un mensaje: "Error: El ID ya está en uso".

```
BEGIN TRY
    -- Intentamos insertar un equipo con un código que ya existe en el script original
    INSERT INTO Equipos (Codigo, Nombre) VALUES ('#RMDRID', 'Real Madrid Duplicado');
END TRY
BEGIN CATCH
    IF ERROR_NUMBER() = 2627 -- Código de error para violación de PK
        PRINT 'Error: El ID ya está en uso';
    ELSE
        PRINT 'Ocurrió un error diferente: ' + ERROR_MESSAGE();
END CATCH;
```

2. Crea una consulta que calcule el "Promedio de Goles por Minuto". Usa TRY-CATCH para capturar el error si un jugador tiene 0 minutos jugados.

```
BEGIN TRY
    SELECT Nombre, Goles / Minutos AS Promedio
    FROM Futbolistas
    WHERE Codigo = 'VL0020'; -- Ferrán Torres tiene minutos, pero si tuviera 0 fallaría
END TRY
BEGIN CATCH
    PRINT 'Error: No se puede calcular el promedio porque el jugador tiene 0 minutos jugados.';
END CATCH;
```

3. Envuelve el ejercicio del "Traspaso entre equipos" (Bloque 3, Ej 1) en un bloque TRY-CATCH. Si entra en el CATCH, ejecuta el ROLLBACK automáticamente.

```
BEGIN TRY
    BEGIN TRANSACTION;
        UPDATE Futbolistas SET CodigoEquipo = '#FCBLNA' WHERE Codigo = 'AB0019';
        -- Forzamos un error si queremos probar (ej. actualizando tabla inexistente)
        UPDATE Equipos SET PJ = PJ + 1 WHERE Codigo = '#FCBLNA';
    COMMIT;
    PRINT 'Traspaso realizado con éxito.';
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT 'Error en el traspaso. Se ha revertido la operación automáticamente.';
END CATCH;
```

4. En el bloque CATCH, inserta los detalles del error (puedes usar funciones como ERROR_MESSAGE()) en una tabla llamada Log_Errores.

```
BEGIN TRY
    SELECT 1/0; -- Forzamos error
END TRY
BEGIN CATCH
    INSERT INTO Log_Errores (Mensaje, Numero)
    VALUES (ERROR_MESSAGE(), ERROR_NUMBER());
    PRINT 'El error ha sido registrado en la tabla Log_Errores.';
END CATCH;
```

5. Intenta insertar un texto en una columna de tipo INT. Captura el error y muestra un mensaje amigable.

```
BEGIN TRY
    -- Intentamos meter texto en la columna Edad (INT)
    INSERT INTO Futbolistas (Codigo, Nombre, Edad) VALUES ('ERR01', 'Error User', 'Treinta años');
END TRY
BEGIN CATCH
    PRINT 'Error de formato: Has intentado introducir texto en un campo numérico. Por favor, revisa los datos.';
END CATCH;
```

6. Crea un TRY-CATCH para capturar cuando un UPDATE falla porque viola una restricción CHECK (ej. edad negativa).

```
BEGIN TRY
    UPDATE Futbolistas SET Edad = -5 WHERE Codigo = 'VL0020';
END TRY
BEGIN CATCH
    PRINT 'Error: La edad no puede ser un valor negativo.';
END CATCH;
```

7. Crea un bloque TRY-CATCH que dentro del CATCH tenga otro TRY-CATCH (por si falla el proceso de loguear el error).

```
BEGIN TRY
    DELETE FROM Equipos; -- Intentamos borrar (fallará por FKs)
END TRY
BEGIN CATCH
    PRINT 'Fallo el borrado, intentando registrar el error...';
    BEGIN TRY
        -- Intentamos insertar en una tabla de log que "podría" no existir
        INSERT INTO TablaLogInexistente (Mensaje) VALUES (ERROR_MESSAGE());
    END TRY
    BEGIN CATCH
        PRINT 'Incluso el registro del error falló. Contacte con el administrador.';
    END CATCH
END CATCH;
```

8. Intenta insertar un NULL en una columna definida como NOT NULL. Captura el error y usa un PRINT para avisar al administrador.

```
BEGIN TRY
    -- En el script, Codigo es PRIMARY KEY (NOT NULL por definición)
    INSERT INTO Futbolistas (Codigo, Nombre) VALUES (NULL, 'Jugador Sin Codigo');
END TRY
BEGIN CATCH
    PRINT 'ATENCIÓN ADMINISTRADOR: Se ha intentado insertar un valor nulo en una columna obligatoria.';
END CATCH;
```